Community detection in hypergraphs through hyperedge percolation: a re-implementation and a label-free parameter rule

Mohamed Aklamaash Luqmaan

October 28, 2026

Abstract

Hyperedge percolation finds communities in hypergraphs without turning them into graphs first. Kovács, Benedek and Palla propose two variants and leave their parameters to the user. We re-implemented both variants with tests and checked them in the synthetic setting of their Figure 1. We then compared them with clique percolation on the pairwise projection when community and noise hyperedges share sizes and noise is heavy. We also tried a simple rule that picks parameters without using the planted labels. The rule keeps settings that give at least two communities, with the largest at most twice the second largest, and picks the one with the highest concentration score. We compare it with the oracle-best setting and with the paper's default on synthetic hypergraphs and on House and Senate committee data. Clique percolation scored 0.100 to 0.105 in every noisy case it finished, in one layout with a clean size cut and two with overlapping sizes, against 0.465 to 1.000 for the better hyperedge method. The rule beat the paper default on synthetic data on average (ECS 0.555 against 0.292 for the large-edge variant, 0.586 against 0.229 for the small-edge variant) but stayed below the oracle (0.644 and 0.698). On the committee data it did not help: the default was as good or better in every case but one, where the rule led by 0.003. Clique percolation became too slow on large hyperedges: 278 of 350 runs timed out, all but 10 of them when community hyperedges had four or more nodes. We list the limits of the rule at the end.

1 Introduction

A hypergraph lets one edge join any number of nodes. This fits groups: a committee, a paper with many authors, a chemical reaction [1]. The usual shortcut flattens the hypergraph into a graph, so a hyperedge of size s becomes $s(s-1)/2$ pairs. This has two costs. Large groups get over-counted, because they produce many more pairs than small groups. And the graph forgets that all those pairs came from one event.

A community is a set of nodes that belong together. A partition gives every node exactly one community. A cover allows overlap, and may leave some nodes out. Clique percolation [5] gives a cover. It takes the k-cliques of a graph, calls two of them adjacent if they share $k-1$ nodes, and returns the node unions of connected groups of cliques.

Kovács, Benedek and Palla [4] carry the same idea to hypergraphs. Communities are built from hyperedges directly. They give two variants. One keeps hyperedges of size at least k and joins two when they share at least I_{abs} nodes. The other keeps hyperedges of size at most K and lets a hyperedge join a growing community when enough of its nodes are already inside. They show that the result can differ from clique percolation on the projection.

Their paper leaves three things open. The parameters (k, I_{abs}, K, I_{rel}) are chosen by hand. The synthetic test has a clean size cut: community hyperedges are all larger (or all smaller) than noise hyperedges. And the comparison with clique percolation on the projection is shown by example, not measured over many instances.

Algorithm 1 Large-edge hyperedge percolation

Input: hypergraph H, minimum size k, absolute overlap I_{abs}
$L \leftarrow$ hyperedges of H with size $\geq k$
join two hyperedges in L if they share $\geq I_{\text{abs}}$ nodes
each connected component of hyperedges is a candidate community: the union of its nodes
drop every candidate that is contained in another candidate
Output: the remaining node sets

Algorithm 2 Small-edge hyperedge percolation

Input: hypergraph H, maximum size K, relative overlap $I_{\text{rel}} \in [0.5, 1)$
$L \leftarrow$ hyperedges of H with size $\leq K$; all unassigned
for all unassigned hyperedge h in L taken as seed **do**
  $C \leftarrow$ nodes of h
  repeat
    add to C the nodes of any hyperedge $e \in L$ of size S with $|e \cap C| \geq \lfloor I_{\text{rel}} \cdot S \rfloor$
  until C no longer changes
  mark the hyperedges inside C as assigned; record C
end for
drop every recorded set that is contained in another
Output: the remaining node sets

We make three contributions.

1. An independent re-implementation of both variants, with tests, checked in the setting of Figure 1 of the paper. We do not claim to reproduce their figures.

2. A measured comparison against clique percolation on the projection when the size cut is not clean and noise is heavy.

3. A simple rule that chooses the parameters without labels. We compare it against the oracle-best setting and against the paper's default.

2 Methodology

2.1 Large-edge hyperedge percolation

The large-edge variant assumes big hyperedges carry the community structure. Algorithm 1 follows the description in [4]. Two kept hyperedges are adjacent if they share at least I_{abs} nodes. With $I_{\text{abs}} = k - 1$ this is the hypergraph version of k-clique adjacency. Adjacency is symmetric, so one union-find pass groups the hyperedges.

2.2 Small-edge hyperedge percolation

The small-edge variant assumes small hyperedges are the strong links. A very large hyperedge should not merge things by itself. It compares a hyperedge with the whole current community, not with single hyperedges. This breaks symmetry, so every hyperedge must be re-checked for each community. Algorithm 2 follows [4].

2.3 Label-free parameter selection

We want to pick (k, I_{abs}) or (K, I_{rel}) without planted labels. Algorithm 3 is our rule. The idea is that a good setting should not shatter the hypergraph into tiny pieces and should not merge it into one giant community. The concentration score is large when a few communities

Algorithm 3 Label-free parameter selection

Input: hypergraph H on n nodes, parameter grid $\mathcal{G}$
for all setting $g \in \mathcal{G}$ **do**
  run the method; record community sizes $s_1 \geq s_2 \geq \dots$ and the membership count $m(v)$ of each node
  reject g if there are fewer than two communities, or if $s_1 > 2s_2$
  otherwise score g by $\sum_c e_c^2/n^2$, with $e_c = \sum_{v \in c} 1/m(v)$
end for
choose the surviving g with the highest score; break ties toward the stricter setting
if none survives, use the paper default: $I_{\text{abs}} = k - 1$ with k the smallest hyperedge size ≥ 3 (large-edge variant), or K the largest hyperedge size with $I_{\text{rel}} = 0.5$ (small-edge variant)

hold most of the nodes. To handle overlap, each node is split equally across the communities that contain it. The effective size of a community c is $e_c = \sum_{v \in c} 1/m(v)$, where $m(v)$ is the number of communities containing v. The score is $\sum_c e_c^2/n^2$. Rejecting settings where the largest community is more than twice the second largest keeps the rule away from the single giant community, which scores highest on concentration. Our first version scored the surviving settings by coverage instead. It failed because many small overlapping communities can still cover every node. Table 2 keeps it as a comparison.

2.4 Baseline: clique percolation on the projection

The baseline is k-clique percolation on the unweighted clique projection. We find the maximal cliques with at least k nodes and merge two when they share at least $k - 1$ nodes. Communities are the unions, with contained ones dropped. A 60 s time limit stops runs that do not finish.

2.5 Synthetic hypergraphs

The generator follows the random modular hypergraphs of [4]. There are N nodes in C equal communities. Each community gets N/C intra-community hyperedges for each size in the intra size set, with nodes drawn inside the community. For each size in the noise size set, round(Np) noise hyperedges are drawn from all nodes, where p is the noise fraction. The paper keeps the two size sets apart. Our extension lets them overlap.

3 Concepts and Evaluation Metrics

3.1 Metrics

Element-centric similarity (ECS) [3] compares two clusterings node by node. For each node it builds an affinity vector over all nodes with personalised PageRank ($\alpha = 0.9$) on the node–community structure, and compares these vectors between the two clusterings. It handles overlap, and it scores 1 for identical clusterings. Unassigned nodes count as singleton communities, as in the paper. **Coverage** is the fraction of nodes in at least one community. **Largest-community fraction** is the size of the biggest community divided by N. We also report the **number of communities**.

3.2 Experimental setup

E1, check. The setting of Figure 1 in [4]: $N = 500$, $C = 10$, intra sizes $\{6, 7, 8\}$ with noise sizes $\{3, 4, 5\}$, and the mirror case. Noise fraction runs from 0 to 0.4. The paper averages 100 realisations per cell. Grids: $k \in 3..8$ with $I_{\text{abs}} \in 1..k - 1$; $K \in 3..8$ with $I_{\text{rel}} \in \{0.5, \dots, 0.9\}$. 30 seeds.



Figure 1: E1 check. The grids show mean ECS over 30 seeds at every parameter setting, with one row per noise level from 0 to 0.4. Left figure: intra sizes 6 to 8, noise sizes 3 to 5. Right figure: intra sizes 3 to 5, noise sizes 6 to 8. In each figure the left grid is the large-edge variant and the right grid the small-edge variant.

E2, map. Five layouts of intra and noise sizes: $(6,7,8)/(3,4,5)$, $(5,6,7)/(4,5,6)$, $(4,5,6)/(4,5,6)$, $(4,5,6)/(5,6,7)$ and $(3,4,5)/(6,7,8)$. Noise is 0, 0.1, 0.2, 0.3, 0.4, 0.6 or 0.8. Each variant is scored at the parameters with the best mean ECS in its cell. Clique percolation uses $k \in \{3,4\}$, 5 seeds and a 60 s limit.

E3, tune. Algorithm 3 against the coverage rule, the oracle and the default, on the 1050 hypergraphs per variant of the E2 grid (30 seeds for each layout and noise level). The oracle here is the best setting for each single hypergraph, so it differs from the per-cell oracle of E2.

E4, real. House and Senate committee hypergraphs with party labels [2]. Pairwise edges are dropped, as in the paper.

4 Results

All values are means over seeds. Each method is scored at its best parameters in E1 and E2, so those numbers show what a method can do at best, not what a user without labels would get.

E1, check. In the large-intra, small-noise case the large-edge variant reaches ECS 1.000 at every noise level from 0 to 0.4. The small-edge variant starts at 1.000 at noise 0 and falls to 0.711, 0.571, 0.482 and 0.424 at noise 0.1, 0.2, 0.3 and 0.4. In the mirror case the order flips. The small-edge variant stays at 1.000 at every noise level, and the large-edge variant falls from 1.000 to 0.400, 0.282, 0.233 and 0.206. So each variant is perfect when its size assumption fits and degrades when it does not (Figure 1).

E2, map. Table 1 and Figures 2–3 give the mean ECS per size layout and noise level. Overlap in the size sets does not break the hyperedge variants by itself. In (5,6,7)/(4,5,6) the large-edge variant scores 0.999 at every noise level from 0.1 to 0.8. In (4,5,6)/(5,6,7) the small-edge variant scores 0.968 to 0.971. The layout with identical sizes, (4,5,6)/(4,5,6), is hard for both: 0.601 and 0.601 at noise 0.1, and 0.238 and 0.248 at noise 0.8 (large-edge and small-edge). Which variant wins follows which side has the larger hyperedges. Clique percolation reached 1.000 at noise 0 in the three layouts where it finished. With any noise it scored 0.100 to 0.105 wherever it finished, which is noise 0.1 to 0.6 in (3,4,5)/(6,7,8), noise 0.1 and 0.2 in (4,5,6)/(4,5,6), and noise 0.1 in

Table 1: E2 mean ECS at oracle-best parameters. Rows are intra/noise size sets at noise levels 0.1 and 0.8. Clique percolation uses the best of k=3 and k=4 over finished runs. n/a: every run timed out or ran out of memory.

Intra / noise	Noise	Large-edge	Small-edge	Clique perc.
(6,7,8) / (3,4,5)	0.1	1.000	0.711	n/a
	0.8	1.000	0.301	n/a
(5,6,7) / (4,5,6)	0.1	0.999	0.640	n/a
	0.8	0.999	0.255	n/a
(4,5,6) / (4,5,6)	0.1	0.601	0.601	0.105
	0.8	0.238	0.248	n/a
(4,5,6) / (5,6,7)	0.1	0.492	0.968	0.102
	0.8	0.189	0.968	n/a
(3,4,5) / (6,7,8)	0.1	0.400	1.000	0.101
	0.8	0.162	1.000	n/a

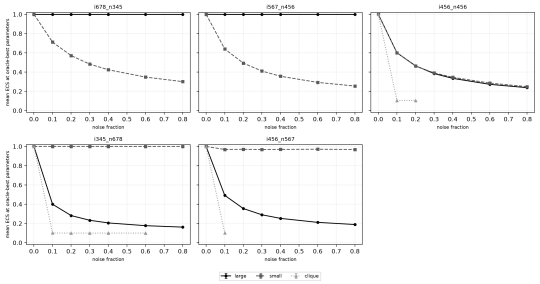


Figure 2: E2 map. The plot shows ECS of each method as noise grows, with one panel per size layout.

(4,5,6)/(5,6,7). In those cells the better hyperedge variant scored 0.465 to 1.000. These clique means use only the runs that finished.

A note on cost: clique percolation on the projection became intractable when community hyperedges had four or more nodes. Runs were stopped at 60 s, and the report also counts runs that ran out of memory as failed. Each layout and k had 35 runs (7 noise levels, 5 seeds). For k=3, 5 runs failed in (3,4,5)/(6,7,8), 34 in (4,5,6)/(4,5,6), 34 in (4,5,6)/(5,6,7), and all 35 in (5,6,7)/(4,5,6) and in (6,7,8)/(3,4,5). For k=4 the counts were 5, 29, 31, 35 and 35. That is 143 of 175 runs for k=3 and 135 of 175 for k=4. Probably the projection of a 50-node community built from size 6 to 8 hyperedges is nearly complete, so the number of maximal cliques explodes. Failed runs are left out of the means. A cell with no finished run shows n/a. The table gives one clique score per cell, the best of k=3 and k=4.

E3, tune. Table 2 compares the rule with the coverage rule, the oracle and the default. Averaged over the 1050 hypergraphs per variant, the rule reaches ECS 0.555 for the large-edge variant and 0.586 for the small-edge variant. The oracle reaches 0.644 and 0.698, and the default 0.292 and 0.229. The rule matches the oracle ECS in 46.6% and 55.4% of hypergraphs. The

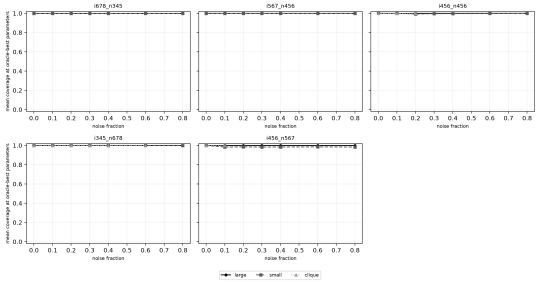


Figure 3: E2 coverage. The plot shows the fraction of nodes placed in a community, for the same layouts as Figure 2.

Table 2: E3 mean ECS of the rule, the coverage rule, the per-hypergraph oracle and the paper default, with the fraction of hypergraphs where each rule reaches the oracle ECS. Each row has 210 hypergraphs per variant. The last rows pool all 1050.

Intra / noise	Variant	Rule	Coverage rule	Oracle	Default	Hit rate	Hit (cov.)
(6,7,8) / (3,4,5)	Large	0.936	0.864	1.000	0.223	0.919	0.000
	Small	0.263	0.363	0.548	0.229	0.143	0.000
(5,6,7) / (4,5,6)	Large	0.983	0.791	0.999	0.263	0.981	0.000
	Small	0.286	0.390	0.493	0.229	0.143	0.000
(4,5,6) / (4,5,6)	Large	0.396	0.449	0.471	0.386	0.143	0.771
	Small	0.410	0.448	0.476	0.229	0.486	0.376
(4,5,6) / (5,6,7)	Large	0.226	0.354	0.399	0.361	0.143	0.067
	Small	0.974	0.529	0.974	0.229	1.000	0.000
(3,4,5) / (6,7,8)	Large	0.233	0.276	0.351	0.229	0.143	0.052
	Small	1.000	0.991	1.000	0.229	1.000	0.719
All layouts	Large	0.555	0.547	0.644	0.292	0.466	0.178
	Small	0.586	0.544	0.698	0.229	0.554	0.219

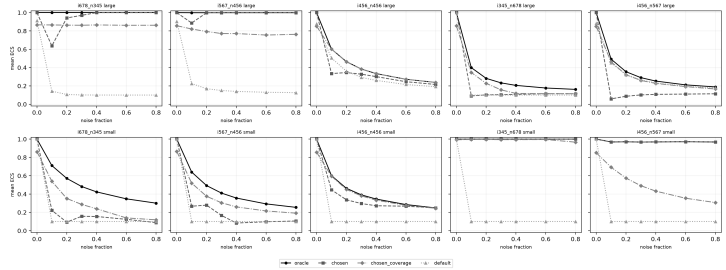


Figure 4: E3 tune. The plot shows mean ECS of the oracle, the rule (chosen), the coverage rule and the default against noise fraction, with one panel per layout and variant.

coverage rule averages 0.547 and 0.544, with hit rates of 17.8% and 21.9%. The rule gains on average over both, but unevenly. It beats the coverage rule in four of ten layout and variant cells and loses in six. It falls below the default in one cell, the large-edge variant on (4,5,6)/(5,6,7), with 0.226 against 0.361. In five cells its hit rate is 0.143, and the per-noise results show that these hits are the noise-0 hypergraphs only. With noise present it never finds the oracle setting in those cells. The fallback was never used (Figure 4).

E4, real. Table 3 lists the results on the committee data. With party labels as reference, the best ECS is 0.501 for the House and 0.500 for the Senate. Both come from settings that return one community holding all nodes, so they say nothing about structure. The rule does not help here. For the House large-edge variant, no setting in the grid passes the giant-community test, so the rule's pick (k=3, I_{abs}=2) is the fallback to the paper default and not a selection. That pick is one community with all 1290 nodes. For the House small-edge variant the rule picks 6 communities (coverage 0.156, ECS 0.026) against 0.501 for the default. For the Senate large-edge variant the rule picks 18 communities with ECS 0.488, a small gain over the default's 0.485 (2 communities). For the Senate small-edge variant it picks 13 communities (coverage 0.525, ECS 0.194) against 0.485 for the default. The coverage rule did no better: 0.032 and 0.169 on the small-edge variants, 0.476 on the Senate large-edge variant and 0.501 on the House large-edge variant. The best values sit near 0.5 and there are only two parties, so we read these numbers with care. Here the default is as good as the rule or better, apart from a gain of 0.003 in one case.

Table 3: E4 committee hypergraphs against party labels. Parameters are (k, I_{abs}) for the large-edge variant and (K, I_{rel}) for the small-edge variant. †Fallback to the paper default: no setting passed the giant-community test.

Data and variant	Setting	Params	Communities	Coverage	Largest frac.	ECS
House, large-edge	Default	(3, 2)	1	1.000	1.000	0.501
	Rule†	(3, 2)	1	1.000	1.000	0.501
	Oracle	(5, 3)	1	1.000	1.000	0.501
House, small-edge	Default	(81, 0.5)	1	1.000	1.000	0.501
	Rule	(12, 0.5)	6	0.156	0.091	0.026
	Oracle	(81, 0.5)	1	1.000	1.000	0.501
Senate, large-edge	Default	(4, 3)	2	1.000	0.947	0.485
	Rule	(14, 10)	18	1.000	0.734	0.488
	Oracle	(14, 1)	1	1.000	1.000	0.500
Senate, small-edge	Default	(31, 0.5)	2	1.000	0.947	0.485
	Rule	(14, 0.7)	13	0.525	0.220	0.194
	Oracle	(31, 0.5)	2	1.000	0.947	0.485

5 Conclusion

On the re-implementation: our code follows the method description in [4], and in their Figure 1 setting the two variants behave as the paper describes. Each variant reaches ECS 1.000 where its size assumption fits and falls (to 0.206 and 0.424 at noise 0.4) where it does not. We did not reproduce their figures.

On the comparison with clique percolation: when community and noise hyperedges share sizes, hyperedge percolation scored higher than clique percolation on the projection in every noisy cell where clique percolation finished (0.100 to 0.105 against 0.465 to 1.000). With identical sizes neither variant does well (0.601 at noise 0.1). Clique percolation also timed out in 278 of 350 runs, all but 10 of them when community hyperedges had four or more nodes.

On the selection rule: on synthetic data the label-free rule beat the paper default on average (0.555 against 0.292, and 0.586 against 0.229) and stayed below the oracle (0.644 and 0.698). It hit the oracle in 46.6% and 55.4% of hypergraphs and fell below the default in one cell. It beat the coverage rule (0.547 and 0.544) in only four of ten cells. On the committee data the rule gave no gain: the default was as good or better, apart from 0.488 against 0.485 on the Senate large-edge variant.

Limits. The synthetic communities are disjoint, so overlap is not tested. The real labels are two parties. Percolation leaves nodes unassigned, which ECS penalises. Our rule is a heuristic, and its giant-community test can reject a correct setting when the true communities differ a lot in size.

References

[1] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. Networks beyond pairwise interactions: structure and dynamics. *Physics Reports*, 874, 2020.

[2] Philip S. Chodrow, Nate Veldt, and Austin R. Benson. Generative hypergraph clustering: from blockmodels to modularity. *Science Advances*, 7, 2021.

[3] Alexander J. Gates, Ian B. Wood, William P. Hetrick, and Yong-Yeol Ahn. Element-centric clustering comparison unifies overlaps and hierarchy. *Scientific Reports*, 9:8574, 2019.

[4] Bianka Kovács, Barnabás Benedek, and Gergely Palla. Community detection in hypergraphs through hyperedge percolation. *Scientific Reports*, 15:36032, 2025. arXiv:2504.15213.

[5] Gergely Palla, Imre Derényi, Illés Farkas, and Tamás Vicsek. Uncovering the overlapping community structure of complex networks in nature and society. *Nature*, 435:814, 2005.